

Practica3.pdf



roro_pocha



Estructura de Computadores



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

70 años formando talento
que transforma el futuro.

La primera escuela de negocios de España,
hoy líder en sostenibilidad y digitalización.



EOI Escuela de
organización
industrial



Descubre EOI

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins?

Plan Turbo: barato

Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
#include <stdio.h> // para printf()
#include <stdlib.h> // para exit()
#include <sys/time.h> // para gettimeofday(), struct timeval
#include <stdint.h> // para exit()

int resultado=0;
#define WSIZE 8*sizeof(long)

#ifndef TEST
#define TEST 5
#endif

/* ----- */
#if TEST==1
/* ----- */
#define SIZE 4
unsigned lista[SIZE]={0x80000000, 0x00400000, 0x00000200, 0x00000001};
#define RESULT 4
/* ----- */
#elif TEST==2
/* ----- */
#define SIZE 8
unsigned lista[SIZE]={0x7fffffff, 0xffbfffff, 0xffffdfff, 0xffffffe,
0x01000023, 0x00456700, 0x8900ab00, 0x00cd00ef};
#define RESULT 156
/* ----- */
#elif TEST==3
/* ----- */
#define SIZE 8
unsigned lista[SIZE]={0x0, 0x01020408, 0x35906a0c, 0x70b0d0e0,
0xffffffff, 0x12345678, 0x9abcdef0, 0xdeadbeef};
#define RESULT 116
/* ----- */
#elif TEST==4 || TEST==0
/* ----- */
#define NBITS 20
#define SIZE (1<<NBITS) // tamaño claro suficiente para tiempo apreciable
unsigned lista[SIZE]; // unsigned para desplazamiento derecha lógico
#define RESULT ( NBITS * ( 1 << NBITS-1 ) ) // pistas para deducir fórmula
/* ----- */
#else
#error "Definir TEST entre 0..4"
#endif
/* ----- */

int popcount1(unsigned* array, size_t len)
{
    int result=0;
    for(size_t j=0; j<len; j++){
        unsigned x = array[j];
        size_t i;
        for (i = 0; i < WSIZE; i++)
        {
```

WUOLAH

```

    result += x & 0x1;
    x >>= 1;
}
}
return result;
}

```

```

int popcount2(unsigned* array, size_t len)
{
    int output=0;
    for(size_t i=0; i<len; i++){
        int result = 0;
        unsigned x = array[i];
        while (x) {
            result += x & 0x1;
            x >>= 1;
        }
        output += result;
    }
    return output;
}

```

```

int popcount3(unsigned* array, size_t len){
    int result = 0;
    for (size_t i=0; i<len; i++) {
        unsigned x = array[i];
        asm("\n"
"ini3: \n\t"           // seguir mientras que x!=0
"shr %[x] \n\t"       // LSB en CF

"adc $0, %[r] \n\t"    // Añade CF a result
"test %[x], %[x] \n\t" // Verifica si x es 0
"jnz ini3 \n\t"       // Si no es 0, vuelve a ini3

: [r]" +r" (result)    // e/salida: añadir a lo acumulado por el momento
: [x] "r" (x);         // entrada: valor elemento
)
    return result;
}

```

```

int popcount4(unsigned* array, size_t len){
    int result = 0;
    for (size_t i=0; i<len; i++) {
        unsigned x = array[i];
        asm("\n"
"clc \n\t"           // CLC para poder empezar por ADC
"ini4: \n\t"         // ahora sale bucle de 3 instrucciones
"adc $0, %[r] \n\t"   // añadir CF a result

"shr %[x] \n\t"       // Desplazamos x a la derecha (LSB en CF)
"jnz ini4 \n\t"       // Si x no es 0, vuelve a ini4
"adc $0, %[r] \n\t"   // Suma el último bit

```

```

        : [r]" + r" (result)           // e/salida: añadir a lo acumulado por el momento
        : [x] "r" (x) );             // entrada: valor elemento
    }
    return result;
}

int popcount5(unsigned* array, size_t len){
    int output = 0;
    for (size_t j=0; j<len; j++){
        unsigned x = array[j];
        int val = 0;
        size_t i;

        for (i = 0; i < 8; i++) {
            val += x & 0x0101010101010101L;
            x >>= 1;
        }

        val += (val >> 16);
        val += (val >> 8);
        output += val & 0xFF;
    }
    return output;
}

int popcount6(unsigned* array, size_t len){
    int output = 0;
    for(size_t i=0; i<len; i++){
        int x = array[i];
        //types and constants used in the functions below
        //uint64_t is an unsigned 64-bit integer type (defined in C99 version of C language)
        const uint32_t m1 = 0x55555555; //binary: 0101...
        const uint32_t m2 = 0x33333333; //binary: 00110011..
        const uint32_t m4 = 0x0f0f0f0f; //binary: 4 zeros, 4 ones ...
        const uint32_t m8 = 0x00ff00ff; //binary: 8 zeros, 8 ones ...
        const uint32_t m16 = 0x0000ffff; //binary: 16 zeros, 16 ones ...
        //This is a naive implementation, shown for comparison,

        x = (x & m1 ) + ((x >> 1) & m1 ); //put count of each 2 bits into those 2 bits
        x = (x & m2 ) + ((x >> 2) & m2 ); //put count of each 4 bits into those 4 bits
        x = (x & m4 ) + ((x >> 4) & m4 ); //put count of each 8 bits into those 8 bits
        x = (x & m8 ) + ((x >> 8) & m8 ); //put count of each 16 bits into those 16 bits
        x = (x & m16) + ((x >> 16) & m16); //put count of each 32 bits into those 32 bits

        output += x;
    }
    return output;
}

int popcount7(unsigned* array, size_t len){
    size_t i;

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins?

Plan Turbo: barato

Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
unsigned x1,x2;  
int result=0;
```

```
const unsigned m1 = 0x55555555; //binary: 0101...  
const unsigned m2 = 0x33333333; // Máscara para cada 4 bits (0011...)  
const unsigned m4 = 0x0F0F0F0F; // Máscara para cada 8 bits (00001111...)  
const unsigned m8 = 0x00FF00FF; // Máscara para cada 16 bits (0000000011111111...)  
const unsigned m16 = 0x0000FFFF; // Máscara para cada 32 bits (16 ceros, 16 unos)
```

```
if (len & 0x3) printf("leyendo 128b pero len no múltiplo de 4n");
```

```
for (i=0; i<len; i+=2){  
    x1 = *(unsigned*) &array[i];  
    x2 = *(unsigned*) &array[i+1];
```

```
    x1 = (x1 & m1) + ((x1 >> 1) & m1); //put count of each 2 bits into those 2 b  
    x1 = (x1 & m2) + ((x1 >> 2) & m2); // Conteo de cada 4 bits  
    x1 = (x1 & m4) + ((x1 >> 4) & m4); // Conteo de cada 8 bits  
    x1 = (x1 & m8) + ((x1 >> 8) & m8); // Conteo de cada 16 bits  
    x1 = (x1 & m16) + ((x1 >> 16) & m16); // Conteo de cada 32 bits
```

```
    x2 = (x2 & m1) + ((x2 >> 1) & m1);  
    x2 = (x2 & m2) + ((x2 >> 2) & m2); // Conteo de cada 4 bits  
    x2 = (x2 & m4) + ((x2 >> 4) & m4); // Conteo de cada 8 bits  
    x2 = (x2 & m8) + ((x2 >> 8) & m8); // Conteo de cada 16 bits  
    x2 = (x2 & m16) + ((x2 >> 16) & m16); // Conteo de cada 32 bits
```

```
    result+= x1+x2;  
}  
return result;  
}
```

```
int popcount8(unsigned* array, size_t len){  
    int i;  
    int val, result = 0;  
    int SSE_mask[] = { 0x0f0f0f0f, 0x0f0f0f0f, 0x0f0f0f0f, 0x0f0f0f0f };  
    int SSE_LUTb[] = { 0x02010100, 0x03020201, 0x03020201, 0x04030302 };
```

```
    if (len & 0x3)  
        printf("leyendo 128b pero len no múltiplo de 4?n");  
    for (i = 0; i < len; i += 4) {  
        asm("movdqu    %[x], %%xmm0 \n\t"  
            "movdqa  %%xmm0, %%xmm1 \n\t" // dos copias de x  
            "movdqu  %[m], %%xmm6 \n\t" // máscara  
            "psrlw   $4, %%xmm1 \n\t"  
            "pand   %%xmm6, %%xmm0 \n\t"//; xmm0 – nibbles inferiores  
            "pand   %%xmm6, %%xmm1 \n\t"//; xmm1 – nibbles superiores
```

```
        "movdqu  %[i], %%xmm2 \n\t"//; ...como pshufb sobrescribe LUT  
        "movdqa  %%xmm2, %%xmm3 \n\t"//; ...queremos 2 copias  
        "pshufb  %%xmm0, %%xmm2 \n\t"//; xmm2 = vector popcount inferiores
```

WUOLAH

```

    "pshufb  %%xmm1, %%xmm3 \n\t"; xmm3 = vector popcount superiores

    "paddb  %%xmm2, %%xmm3 \n\t"; xmm3 = vector popcount bytes
    "pxor   %%xmm0, %%xmm0 \n\t"; xmm0 = 0,0,0,0
    "psadbw %%xmm0, %%xmm3 \n\t"; xmm3 = [pcnt bytes0..7|pcnt bytes8..15]
    "movhps %%xmm3, %%xmm0 \n\t"; xmm3 = [      0      |pcnt bytes0..7 ]
    "paddb  %%xmm3, %%xmm0 \n\t"; xmm0 = [ no usado  |pcnt bytes0..15]
    "movd   %%xmm0, %[val] \n\t"
    : [val]"=r" (val)
    : [x] "m" (array[i]),
      [m] "m" (SSE_mask[0]),
      [l] "m" (SSE_LUTb[0])
    );
    result += val;
}
return result;
}

```

```

// Versión SSE4.2 (popcount)
int popcount9(unsigned* array, size_t len){
    size_t i;
    unsigned x;
    int val, result=0;

    for (i=0; i<len; i++){
        x = array[i];
        asm("popcnt %[x], %[val]"

        : [val] "=r" (val)
        : [x] "r" (x)

        );
        result += val;
    }
    return result;
}

```

```

// popcount 128bit p/mejorar prestaciones
int popcount10(unsigned* array, size_t len){
    size_t i;
    unsigned long x1,x2;
    long val; int result=0;
    if (len & 0x3)
        printf("leyendo 128b pero "
        "len no múltiplo de 4\n");
    for (i=0; i<len; i+=4) {
        x1 = *(unsigned long*) &array[i];
        x2 = *(unsigned long*) &array[i+2];
        asm("popcnt %[x1], %[val] \n"    //altern.
        "popcnt %[x2], %%r10 \n"        //usando
        "add %%r10, %[val] \n"          //lea(),
        : [val]"=&r" (val)

```

PRIMERA CACERÍA
ÚLTIMA OPORTUNIDAD

A movie poster for 'Predator: Badlands'. The central image shows a Predator in its iconic mask and camouflage gear, holding a woman with blonde hair. The Predator is holding a glowing red plasma blade. The woman is holding a large black handgun. The background is a dark, misty jungle with a full moon and birds in flight.

PREDATOR BADLANDS

YA EN CINES

COMPRAR ENTRADAS



PENDIENTE DE CALIFICACIÓN POR EDADES.

```

: [x1] "r" (x1),
[x2] "r" (x2)
);
result += val;
}
return result;
}

```

```

void crono(int (*func)(), char* msg){
    struct timeval tv1,tv2;          // gettimeofday() secs-usecs
    long tv_usecs;                   // y sus cuentas

    gettimeofday(&tv1,NULL);
    resultado = func(lista, SIZE);
    gettimeofday(&tv2,NULL);

    tv_usecs=(tv2.tv_sec -tv1.tv_sec )*1E6+
              (tv2.tv_usec-tv1.tv_usec);
    #if TEST==0
        printf( "%ld" "\n", tv_usecs);
    #else
        printf("resultado = %d\t", resultado);
        printf("%s:%9ld us\n", msg, tv_usecs);
    #endif
}

```

```

int main(){
    #if TEST==0 || TEST==4
        size_t i;                // inicializar array
        for (i=0; i<SIZE; i++)
            lista[i]=i;
    #endif
    crono(popcount1 , "popcount1 (lenguaje C - for)");
    crono(popcount2 , "popcount2 (lenguaje C - while)");
    crono(popcount3 , "popcount3 (leng.ASM-body while 4i)");
    crono(popcount4 , "popcount4 (leng.ASM-body while 3i)");
    crono(popcount5 , "popcount5 (CS:APP2e 3.49-group 8b)");
    crono(popcount6 , "popcount6 (Wikipedia- naive - 32b)");
    crono(popcount7 , "popcount7 (Wikipedia- naive -128b)");
    crono(popcount8 , "popcount8 (asm SSE3 - pshufb 128b)");
    crono(popcount9 , "popcount9 (asm SSE4- popcount 32b)");
    crono(popcount10, "popcount10(asm SSE4- popcount128b)");
    #if TEST != 0
        printf("calculado = %d\n", RESULT);
    #endif
    exit(0);
}

```